

Multimodal Medical Assistant for Image-Text Clinical Triage

HPPCS[04] Capstone Report

Abstract

This project implements a clinician-facing multimodal assistant that combines a radiology scan or histopathology slide with a structured clinical note to produce triage guidance, image–text correlation, evidence-linked summaries, and interactive follow-up chat. The clinical aim is to reduce missed cross-modal cues by forcing vision and text models to share one case record. Compact local models are used deliberately: MedGemma 4B for vision and Llama 3.2 (3B, with 1B fallback) for text reasoning. These sizes were chosen for memory efficiency so the stack runs on a typical 16 GB CPU laptop or on Google Colab (CPU or free T4 GPU), reducing cost, energy use, and data movement compared with large cloud-only multimodal APIs. Ollama auto-adapts to CPU or GPU; the app unloads the vision model after analysis so both LLMs share limited RAM/VRAM. Modalities are restricted to radiology and pathology with auto domain detection. LangGraph sequences analysis and follow-up re-triage. Uploads are de-identified before storage. Evaluation metrics appear in the TUI and GUI after every upload. Dual interfaces share this backend.

1. Introduction

- Context and background: Imaging and notes are usually read in separate tools. This system binds them in one LangGraph state so triage uses both visual findings and vitals or history.
- Motivation: Local MedGemma + Llama on Ollama keep PHI on-premises and match the HPPCS[04] multimodal brief.

2. Problem Statement

Given one radiology or pathology image and one clinical note, the system must (i) detect domain, (ii) correlate image findings with the note, (iii) assign triage (emergency / urgent / soon / routine), (iv) answer clinician follow-up questions with updated recommendations, and (v) report evaluation and interface metrics. Out-of-scope photographs (dermatology, fundus, clinical snapshots) must be rejected.

3. Objectives

- Provide a TUI and a GUI so clinicians can upload a note and image, ask questions, and receive triage summaries.
- Return image-referenced findings, note context, and numbered guideline citations.

- Support adaptive follow-up with Llama re-triage of impression, differential, recommendations, and next questions.
- Auto-detect radiology versus pathology and reject out-of-scope images.
- De-identify text and images before any model call or disk write.
- Emit evaluation metrics (correlation, robustness, explanation quality, satisfaction) on every upload.

4. Methodology

- Tools and technologies: Python 3.12, FastAPI/Uvicorn, LangGraph, Ollama HTTP API, MedGemma 4B (vision), Llama 3.2 3B (text; 1B fallback), runtime_profile.py, python-docx.
- Workflow: ingest_pair de-identifies files → medgemma_analyze (domain + findings) → llama_entities → llama_reason (triage JSON) → llama_converse (cited summary) → optional chat (refresh_reasoning_after_follow_up). Metrics by evaluation.py.

Conceptual pipeline: Note+Image → Privacy gate → MedGemma → Llama entities → Llama triage → Clinician summary → Follow-up re-triage → Evaluation panel in TUI/GUI.

5. System Design / Implementation

Architecture (layers): Presentation (tui_app.py, upload_app.py) → Application (medical_assistant.py sessions) → Orchestration (graph_pipeline.py LangGraph) → Models (ollama_client.py load/unload) → Data (ingestion.py, privacy.py, entity_normalizer.py, clinical_references.py, evaluation.py). sample_data/ sits outside Codebase; live sessions are Codebase/uploads/processed/<id>/.

- main.py — Mode 1 TUI / Mode 2 GUI; --device auto|cpu|gpu; Ollama model checks.
- runtime_profile.py — detects NVIDIA GPU vs CPU; tunes keep-alive, timeouts, unload policy.
- graph_pipeline.py — four-node graph; follow-up JSON re-triage and citations.
- evaluation.py — correlation, robustness, explanation, and satisfaction metrics.
- upload_app.py / tui_app.py — dashboard and terminal chat; metrics panel in both.

6. Observations and Results

Evaluation is performed during a live run through Mode 1 (TUI) or Mode 2 (GUI): upload, analysis, follow-up chat, and the Evaluation panel. Faculty or clinicians may supply their own de-identified note and image at demo time.

Qualitative observations on representative radiology and pathology uploads include: MedGemma correctly distinguishes radiology versus pathology domains; Llama produces structured triage (emergency / urgent / soon / routine) with impression, differential, and recommendations; image-note

correlation text links visual findings to note context; `clinical_references.py` attaches WHO/NIH-style guideline links; and follow-up questions trigger re-triage with updated recommendations and cited replies.

Quantitative results are computed by `evaluation.py` on each upload and shown in the Evaluation panel:

- Cross-modal correlation — presence of an image–note narrative and token overlap between findings and note.
- Robustness — valid domain detection, de-identified storage, and completeness of required output fields.
- Explanation quality — checklist over impression, rationale, findings, correlation, recommendations, references, and follow-up prompts (mapped to a 1–5 Likert).
- User satisfaction — interface completeness proxy from triage, findings, chat, prompt chips, and guideline links (1–5 Likert).

For demonstration, run `python main.py`, choose TUI or GUI, upload one radiology or pathology case, and read the four scores in the Evaluation section alongside triage output and follow-up chat.

7. Conclusion and Future Work

- Summary of contributions: dual-LLM multimodal triage with auto domain detection, privacy gate, dual clinician interfaces, adaptive cited follow-up, and evaluation metric outputs.
- Possible extensions: larger GPU serving when hardware allows, reader-study Likert collection from clinicians, and SNOMED coding via `entity_normalizer.py`.

9. References

- HPPCS[04] capstone brief.
- Google Health AI Developer Foundations, MedGemma model card.
- Meta, Llama 3.2 model card; Ollama library (`medgemma`, `llama3.2`).
- LangGraph documentation, <https://langchain-ai.github.io/langgraph/>
- WHO/NIH public guideline pages linked from `clinical_references.py`.
- Wikimedia Commons teaching images; optional demo files in `sample_data/`.

Acknowledgement

Thanks to HAAI++ faculty for the HPPCS[04] brief and to the MedGemma, Llama, LangGraph, and Ollama communities. Teaching images are public; accompanying notes are fictional.